

Com triar un llenguatge i un framework per al backend?

Criteris professionals de decisió tècnica · Mòdul: Desenvolupament Web en Entorn Servidor · 2n CFGS DAW

DECISIÓ TÈCNICA

BACKEND

ARQUITECTURA



No triem una tecnologia: prenem una decisió tècnica

En un projecte professional, la tecnologia no es tria per preferència personal. Es decideix en funció del context.

Les preguntes clau

- **Funcionalitat**
Què ha de fer l'aplicació?
- **Càrrega**
Quantes peticions simultànies ha de suportar?
- **Dades**
Quin volum i quina sensibilitat?
- **Equip**
Quina experiència té? Quantes persones?

Més factors determinants

- **Temps**
Quant tenim per desenvolupar i per mantindre?
- **Cost**
Llicències, infraestructura, formació.
- **Integracions**
Amb quins sistemes externs ha de comunicar-se?
- **Seguretat**
Quins requisits legals i tècnics existeixen?

❏ **Idea clau:** La tecnologia és una ferramenta. La decisió correcta depén del problema.

Llenguatge, runtime, llibreria i framework: no són el mateix



Llenguatge de programació

Sistema per expressar instruccions.

Ex: JavaScript, PHP, Python, Java, C#



Runtime / entorn d'execució

Programari que executa el codi.

Ex: Node.js (JS), JVM (Java), .NET Runtime (C#)



Llibreria

Conjunt de funcionalitats que el nostre codi crida quan les necessita.

Tu controles el flux.



Framework

Estructura que organitza l'aplicació i crida el nostre codi en determinats moments.

El framework controla part del flux.

Inversió de control

- Amb una **llibreria**, tu la crides.
- Amb un **framework**, moltes vegades és el framework qui et crida a tu.

Exemples per ecosistema

- **Laravel** → PHP
- **Django / FastAPI** → Python
- **Express / NestJS** → JS/TS
- **Spring Boot** → Java
- **ASP.NET Core** → C#

Framework minimalista vs framework complet

MINIMALISTA

Ex: Express

Proporciona una base mínima: routing i middleware. L'equip decideix arquitectura, ORM, autenticació, validació i estructura.

✓ Avantatges

- Flexibilitat total
- Adequat per a serveis menuts

❌ Inconvenients

- Més decisions a prendre
- Risc d'arquitectura inconsistent

COMPLET / OPINIONATED

Ex: Laravel, Django, NestJS, Spring Boot

Incorpora: routing, ORM, autenticació, validació, testing, CLI, migracions, injecció de dependències.

✓ Avantatges

- Convencions establertes
- Menys decisions repetitives
- Incorporació més fàcil de nous membres

❌ Inconvenients

- Cal aprendre les convencions del framework

📌 **Important:** "Opinionated" no significa pitjor. Significa que proporciona decisions ja preses que acceleren el desenvolupament.

Primera pregunta: quin tipus de producte estem construint?



Aplicació web tradicional

Backend que genera HTML. Necessita plantilles, sessions, formularis, ORM i autenticació integrada.



API REST

Exposa dades i funcionalitats. Prioritza routing, serialització JSON, validació, autenticació i documentació OpenAPI.



Microservei

Responsabilitat única i reduïda. Prioritza baix consum de recursos, temps d'arrancada ràpid, observabilitat i comunicació entre serveis.



Aplicació en temps real

Xat, notifikacions, seguiment en directe. Requereix WebSockets, moltes connexions simultànies i model d'execució asíncron.



Idea clau: Primer definim el problema. Després comparem tecnologies.

Rendiment: què significa realment?

"Rendiment" no és un valor únic. Depén del que mesurem:

Latència

Temps de resposta per petició

Throughput

Peticions per segon

Memòria RAM

Consum en repòs i sota càrrega

Ús de CPU

Processament per petició

Connexions simultànies

Capacitat de concurrència

CPU-BOUND

Tasques que consumeixen sobretot processador: compressió, xifrat intensiu, processament d'imatges, càlcul matemàtic.

Aquí el rendiment brut del llenguatge importa més.

I/O-BOUND

Tasques que esperen recursos externs: consultes a BD, crides a APIs, lectura de fitxers.

- ❏ **La majoria d'aplicacions web són principalment I/O-bound.** Un llenguatge teòricament més ràpid no garanteix que tota l'aplicació siga més ràpida.

Síncron, asíncron i concurrència

Execució síncrona

Una operació espera que l'anterior acabe: consultar BD → esperar → continuar. Simple de llegir, però pot bloquejar el fil d'execució.

Execució asíncrona

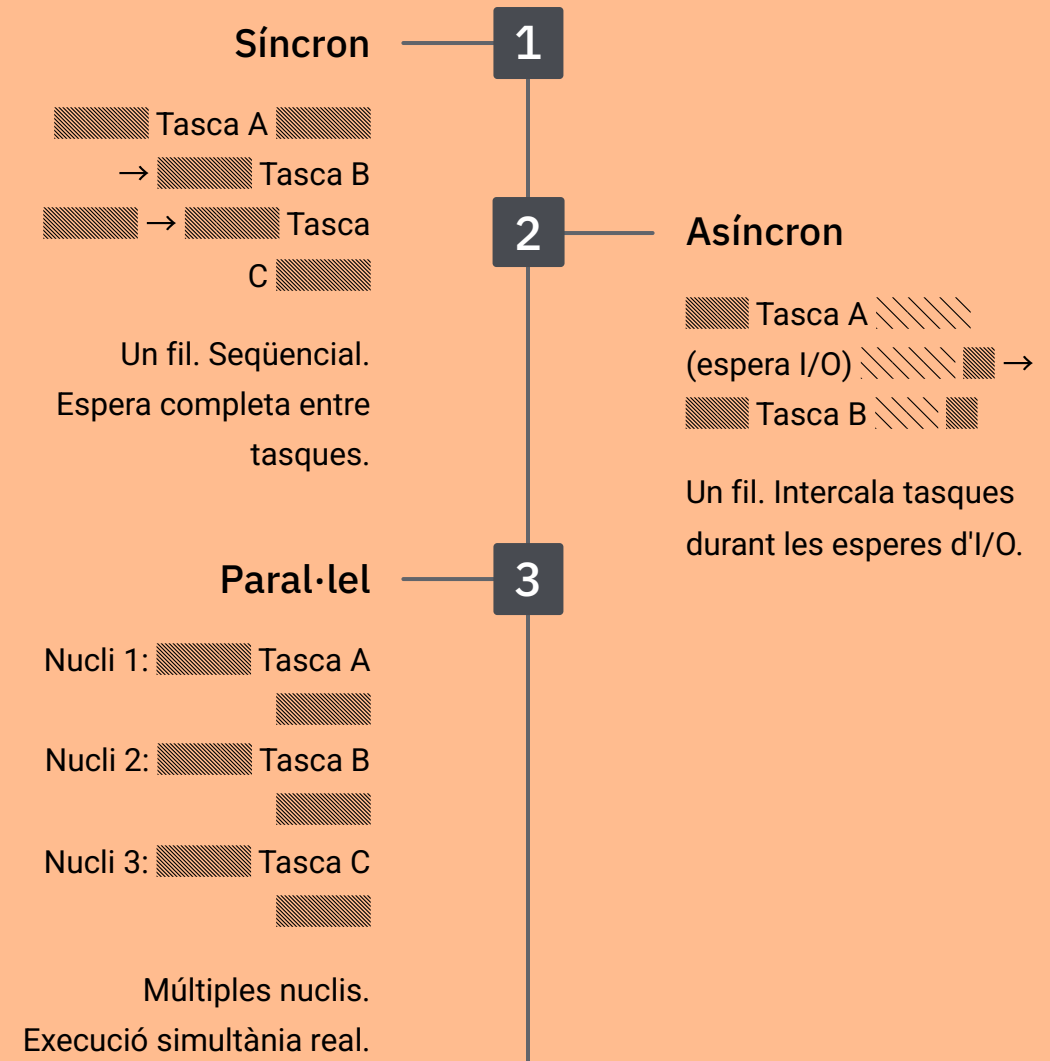
El programa inicia una operació lenta i continua gestionant altres tasques mentre espera. Especialment útil en aplicacions I/O-bound.

Concurrència vs Paral·lisme

Concurrència: Gestionar diverses tasques durant el mateix interval de temps (no necessàriament al mateix instant).

Paral·lisme: Diverses tasques s'executen literalment al mateix temps, usant diversos nuclis de CPU.

Diagrama de línies de temps



Asíncron, concurrent i paral·lel no són exactament el mateix. Un sistema pot ser concurrent sense ser paral·lel.

Escalabilitat: vertical i horitzontal

ESCALABILITAT VERTICAL

Fer més potent una màquina

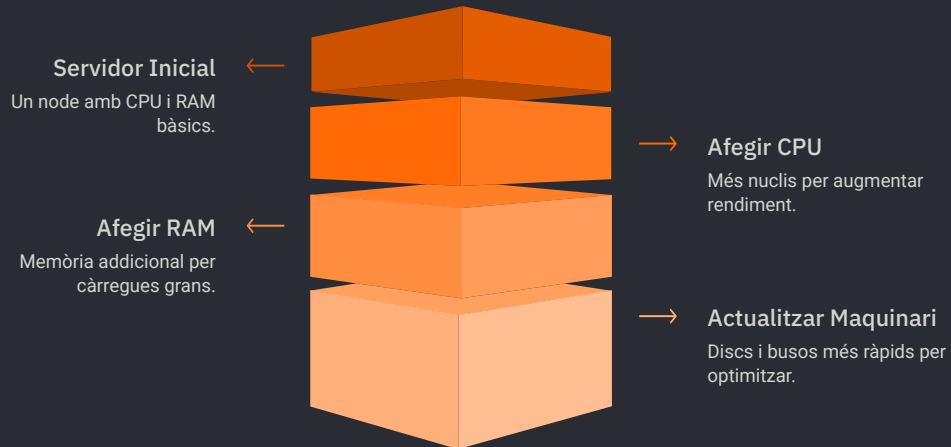
Més CPU, més RAM, millor maquinari.

✓ Avantatge

Fàcil d'implementar

⚠ Límit

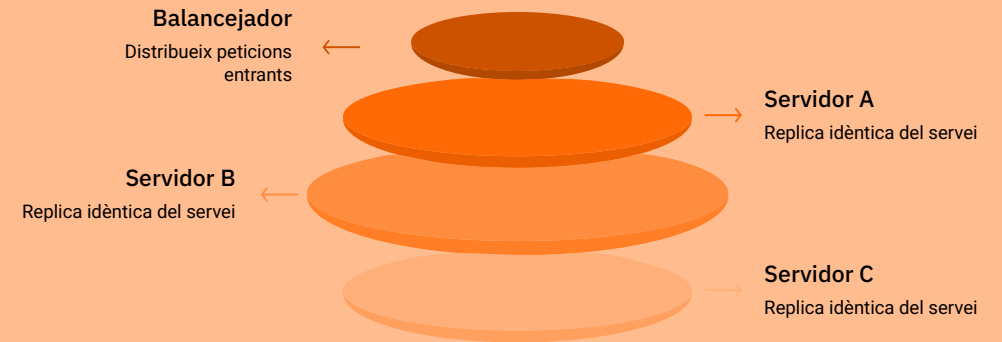
Té un sostre físic i econòmic



ESCALABILITAT HORITZONTAL

Afegir més màquines o instàncies

1 servidor → 3 servidors → balancejador de càrrega. Requereix que l'aplicació siga **stateless** o quasi stateless.



Concepte: Stateless

L'estat de sessió es guarda en un lloc compartit (BD, Redis, token JWT), no en la memòria del servidor. Permet que qualsevol instància pugui atendre qualsevol petició.

Ecosistema: sovint és més important que el llenguatge

Quan triem una tecnologia, triem tot el seu ecosistema:



Llibreries i paquets

Disponibilitat i maduresa per als casos d'ús del projecte.



ORM i connectors de BD

Qualitat i compatibilitat amb els motors que necessitem.



Eines de testing

Frameworks de proves, mocks, fixtures.



Documentació

Qualitat, actualització i idiomes disponibles.



Comunitat

Fòrums, Stack Overflow, GitHub Issues, tutorials.



Suport cloud

AWS, GCP, Azure, Vercel, Railway, Fly.io.

📌 **Idea clau:** Una tecnologia pot ser excel·lent tècnicament, però inadequada si no existeixen llibreries madures per als serveis que necessita el projecte.

Maduresa i manteniment

La popularitat actual no és l'únic indicador. Cal observar:

Anys d'evolució

Quant de temps porta en producció real?

Freqüència d'actualitzacions

Es corregeixen errors i vulnerabilitats amb rapidesa?

Qualitat de la documentació

Completa, actualitzada, amb exemples reals?

Versions LTS

Long-Term Support: versions que reben suport durant un període prolongat. Especialment importants en projectes empresarials que poden durar anys.

Estabilitat de les APIs

Canvien sovint i trenquen compatibilitat?

Nombre de mantenidors

Depén d'una sola persona o d'una organització?

Resposta davant vulnerabilitats

Publiquen CVEs? Parchegen ràpid?

Un framework amb 10 anys d'historial, versions LTS i una fundació darrere ofereix garanties molt diferents d'un projecte d'una sola persona amb l'última versió fa 2 anys.

Corba d'aprenentatge vs productivitat

Facilitat inicial

- Quant tarda una persona en construir alguna cosa funcional?
- Alguns frameworks permeten un prototip en hores. Altres requereixen dies de configuració.

Productivitat a mitjà termini

- Quant costa afegir funcionalitats noves?
- Quant tarda a incorporar-se un nou desenvolupador?
- Quant costa corregir errors i provar canvis?
- Quant costa mantindre el projecte en 2 anys?

Convenció sobre configuració

Alguns frameworks assumeixen convencions: on estan els controladors, com es nomenen els fitxers, com s'estructuren els models.

✓ Avantatge

Menys decisions repetitives, consistència entre projectes.

❌ Inconvenient

Cal aprendre les convencions del framework.

📌 **Idea:** En projectes grans, la consistència arquitectònica pot ser més important que escriure poc codi.

Seguretat

La seguretat no depén només del llenguatge. Cal valorar si el framework ajuda amb:

Autenticació i autorització

Mecanismes integrats o llibreries oficials?

Gestió de sessions

Segura per defecte?

Protecció CSRF

Activada automàticament?

Validació d'entrada

Facilita la sanitització de dades?

SQL Injection i XSS

Protegit per defecte via ORM i escapament?

Gestió de contrasenyes i secrets

Hashing, variables d'entorn, vaults?

Actualitzacions de seguretat

Freqüència i rapidesa de resposta?

Secure by default

Un framework és més segur per defecte quan les seues configuracions habituals eviten molts errors comuns sense que el programador haja d'implementar totes les proteccions manualment.

 **Però:** Cap framework elimina la responsabilitat del desenvolupador.

Mantenibilitat i deute tècnic

La major part de la vida d'un programa no consisteix a crear-lo, sinó a mantindre'l.

Una tecnologia adequada hauria de facilitar:

01

Llegibilitat

Llegir codi escrit fa mesos per una altra persona.

02

Modificabilitat

Modificar funcionalitats sense trencar altres parts.

03

Testabilitat

Provar canvis amb confiança.

04

Detecció d'errors

Detectar errors ràpidament.

05

Actualitzabilitat

Actualitzar dependències sense regressions.

06

Incorporació

Incorporar nous membres a l'equip.

Deute tècnic

Decisions ràpides que faciliten el desenvolupament ara, però que generen més cost en el futur.

Causes habituals

- Codi difícil de llegir
- Sense tests automatitzats
- Acoblament fort entre components

📌 El deute tècnic no és sempre evitable, però cal ser conscient

Testabilitat

Cal valorar si una tecnologia facilita les proves. Si és difícil de provar, serà difícil de mantindre amb confiança.

Tipus bàsics de proves

01

Proves unitàries

Comproven una funció o classe de manera aïllada.

02

Proves d'integració

Comproven que diverses parts del sistema funcionen juntes.

03

Proves funcionals / end-to-end

Simulen el comportament real de l'usuari.

Què hauria de facilitar un bon ecosistema backend?

Executar tests de manera senzilla i ràpida.

Simular dependències (mocks, stubs, fakes).

Preparar bases de dades de prova (fixtures, factories).

Automatitzar proves en pipelines CI/CD.

Mesurar la cobertura de codi.

Integració amb bases de dades i ORM

Aspectes a valorar



Motors suportats

PostgreSQL, MySQL, SQLite, MongoDB, Redis...



Qualitat del driver

Estable, mantingut, amb suport de transaccions?



Migracions

Permet gestionar l'evolució de l'esquema de BD?



Pooling de connexions

Gestiona eficientment les connexions obertes?

ORM: Object-Relational Mapping

Definició

Representa dades de bases de dades relacionals mitjançant objectes o entitats del llenguatge.

Avantatges

Menys SQL repetitiu, major productivitat, integració amb el model de l'aplicació.

Riscos

Consultes ineficients, desconeixement del SQL real generat, problemes de rendiment si s'usa sense entendre el que passa internament.



Idea important: Utilitzar un ORM no elimina la necessitat d'entendre les bases de dades.

Arquitectura, acoblament i injecció de dependències

Acoblament

Una part del sistema està molt acoblada a una altra quan depèn fortament d'ella.

Molt acoblament dificulta:

Canviar components

Fer proves

Reutilitzar codi

Migrar tecnologies

Mecanismes dels frameworks

01

Capes: controladors, serveis, repositoris.

02

Interfaces i abstraccions.

03

Injecció de dependències.

Injecció de dependències

En lloc que un objecte cree directament tot el que necessita, ho rep des de fora.

Facilita substituir components (ex: canviar la BD de prova per una real).

Facilita les proves unitàries: podem injectar mocks.

Experiència de desenvolupament (DX)

Developer Experience: la qualitat del procés de desenvolupament dia a dia.



IDE i autocompletat

Suggeriments precisos, navegació per codi, refactorització.



CLI

Generadors de codi, scaffolding, migracions des de la terminal.



Depurador

Permet inspeccionar l'estat del programa en temps d'execució.



Documentació

Completa, amb exemples reals i actualitzada.



Hot reload

Veure canvis sense reiniciar el servidor.



Tipatge estàtic

Detecta determinats errors abans d'executar el programa. Millora l'autocompletat i la refactorització. No és una superioritat absoluta: depén del projecte i l'equip.



Missatges d'error

Clars, amb context i suggeriments de solució.

Desplegament i infraestructura

Una tecnologia s'ha d'avaluar pensant en producció, no només en desenvolupament.



Contenidors Docker

És fàcil crear una imatge lleugera i reproducible?



Compatibilitat cloud

AWS Lambda, Google Cloud Run, Azure App Service, Vercel, Railway...



Consum de memòria

Quanta RAM necessita en repòs i sota càrrega?



Serverless

Funciona bé amb execucions curtes i sense estat persistent?



Temps d'arrancada

Importa especialment en serverless i en escalat horitzontal ràpid.



Monitorització

Integra fàcilment amb Prometheus, Datadog, Sentry, OpenTelemetry?



Servidor d'aplicacions

Necessita Nginx, Apache, Gunicorn, Tomcat...?

 **Idea:** Desenvolupament i producció formen part del mateix problema. Una tecnologia difícil de desplegar genera costos operatius continus.

Mercat laboral i equip

Un projecte no és només tecnologia. Les persones que el desenvolupen i mantenen són part de la decisió.

Coneixement actual de l'equip

Quant temps cal per ser productiu?

Experiència interna

L'empresa ja té projectes en aquesta tecnologia?

Facilitat per contractar

Hi ha professionals disponibles al mercat local?

Comunitat local

Meetups, conferències, grups d'usuaris?

Disponibilitat de formació

Cursos, bootcamps, certificacions oficials?

Exemple: Una tecnologia pot ser tècnicament excel·lent però resultar poc viable si ningú de l'equip sap mantindre-la i és difícil contractar professionals.

El factor IA en el desenvolupament

Actualment té sentit valorar com funciona una tecnologia en entorns de desenvolupament assistit per IA.



Quantitat de codi públic

Més codi públic → millors suggeriments dels models.



Facilitat per generar tests

Frameworks amb estructura clara → tests generats més fiables.



Qualitat de la documentació

Documentació clara → millors respostes de l'IA.



Sistema de tipus

Tipatge estàtic → l'IA pot verificar millor el codi generat.



Patrons ben establerts

Convencions conegudes → codi generat més consistent.



Eines d'anàlisi estàtica

Linters, type checkers → detecció d'errors en codi generat.



Idea important: Que una IA genere molt bé codi d'una tecnologia no significa que siga la tecnologia adequada. La decisió tècnica continua requerint criteri humà.

Comparativa orientativa de frameworks backend

Valors orientatius. Depenen del context, l'equip i el tipus de projecte.

Criteri	Laravel	Express	NestJS	Django / FastAPI	Spring Boot	ASP.NET Core
Corba d'aprenentatge	Mitjana	Baixa	Mitjana-alta	Baixa-mitjana	Alta	Mitjana-alta
Estructura imposada	Alta	Baixa	Alta	Mitjana-alta	Alta	Alta
Ecosistema	Alt	Alt	Mitjà-alt	Alt	Molt alt	Alt
APIs REST	Bona	Bona	Excel·lent	Excel·lent	Bona	Excel·lent
Aplicacions grans	Bona	Limitada	Excel·lent	Bona	Excel·lent	Excel·lent
Prototipat ràpid	Alt	Alt	Mitjà	Alt	Baix	Mitjà
Tipatge	No (PHP)	No (JS)	Sí (TS)	Parcial / Sí	Sí (Java)	Sí (C#)
Integració empresarial	Bona	Limitada	Bona	Bona	Excel·lent	Excel·lent
Productivitat inicial	Alta	Alta	Mitjana	Alta	Baixa-mitjana	Mitjana

Una possible matriu de decisió

En un projecte professional podem puntuar cada opció tecnològica per criteris ponderats.

Exemple de criteris i pesos

1	Seguretat × 5 Requisit crític en la majoria de projectes.
2	Mantenibilitat × 5 El codi viurà anys.
3	Ecosistema × 4 Llibreries, integracions, comunitat.
4	Rendiment × 3 Depén molt del tipus d'aplicació.
5	Experiència de l'equip × 4 Impacte directe en productivitat.

1	Testing × 3 Facilitat per provar el sistema.
2	Desplegament × 3 Cost operatiu en producció.
3	Mercat laboral × 2 Facilitat per contractar.
4	Popularitat × 1 Indicador, no determinant.



Idea clau: No tots els criteris tenen la mateixa importància. Els pesos canvien segons el projecte.

Error habitual: comparar només benchmarks

Un benchmark mesura una situació molt concreta i controlada.

Què mesura un benchmark?

Normalment mesura:
peticions per segon en un
endpoint mínim, sense lògica
de negoci real.

Els benchmarks poden ser
útils per comparar rendiment
brut en casos específics (CPU-
bound, alta concurrència).

Què NO mesura?

Qualitat del codi

Productivitat de l'equip

Mantenibilitat

Seguretat

Cost del desenvolupament

Facilitat de contractació

Rendiment real amb ORM, validació, autenticació i lògica de negoci

Conclusió: 'A és més ràpid que B en un benchmark' no implica 'A és millor que B per al teu projecte'.

Error habitual: triar per moda

Les modes tecnològiques canvien. Abans d'adoptar una tecnologia nova, cal preguntar:

Resol un problema real que la tecnologia actual no resol bé?

És estable? Té una API estable o canvia sovint?

Existeix comunitat? Hi ha respostes a Stack Overflow, GitHub Issues actius?

Té documentació? Completa i actualitzada?

Sabem mantindre-la? L'equip pot aprendre-la en el temps disponible?

Tindrà suport d'ací a uns anys? Qui hi ha darrere? Empresa, fundació, individu?

 **La novetat no és un avantatge per si mateixa en un context professional.**

Error habitual: triar sempre el que coneixem

El problema contrari també existeix.

Utilitzar sempre la mateixa tecnologia evita aprendre i pot generar solucions inadequades per al problema.

Canviar de tecnologia constantment genera costos: formació, reescriptura, risc.

Concepte: 'Boring technology'

Una tecnologia madura, coneguda i poc innovadora pot ser una decisió excel·lent si resol bé el problema.

PostgreSQL, Linux, PHP, Java: tecnologies "avorrida" que alimenten gran part d'internet.

La innovació té valor, però també té cost. Cal saber quan val la pena.

 **Equilibri:** Ni sempre el mateix, ni canviar per canviar. La decisió ha de ser justificada.

Les preguntes que hem de saber respondre

Abans de triar una tecnologia backend, un professional ha de poder respondre:

01

Quin problema volem resoldre?

02

Quin tipus d'aplicació estem construint?

03

Quina càrrega esperem?

04

Quines integracions necessitem?

05

Quin nivell de seguretat necessitem?

06

Quina experiència té l'equip?

07

Com provarem l'aplicació?

08

Com la despleguem?

09

Qui la mantindrà?

10

Quant de temps haurà de viure?

11

Quin ecosistema necessitem?

12

Quin cost tindrà canviar de tecnologia en el futur?

Un bon desenvolupador no és qui coneix 'el millor framework'. És qui sap justificar per què una tecnologia és adequada per a un problema concret.